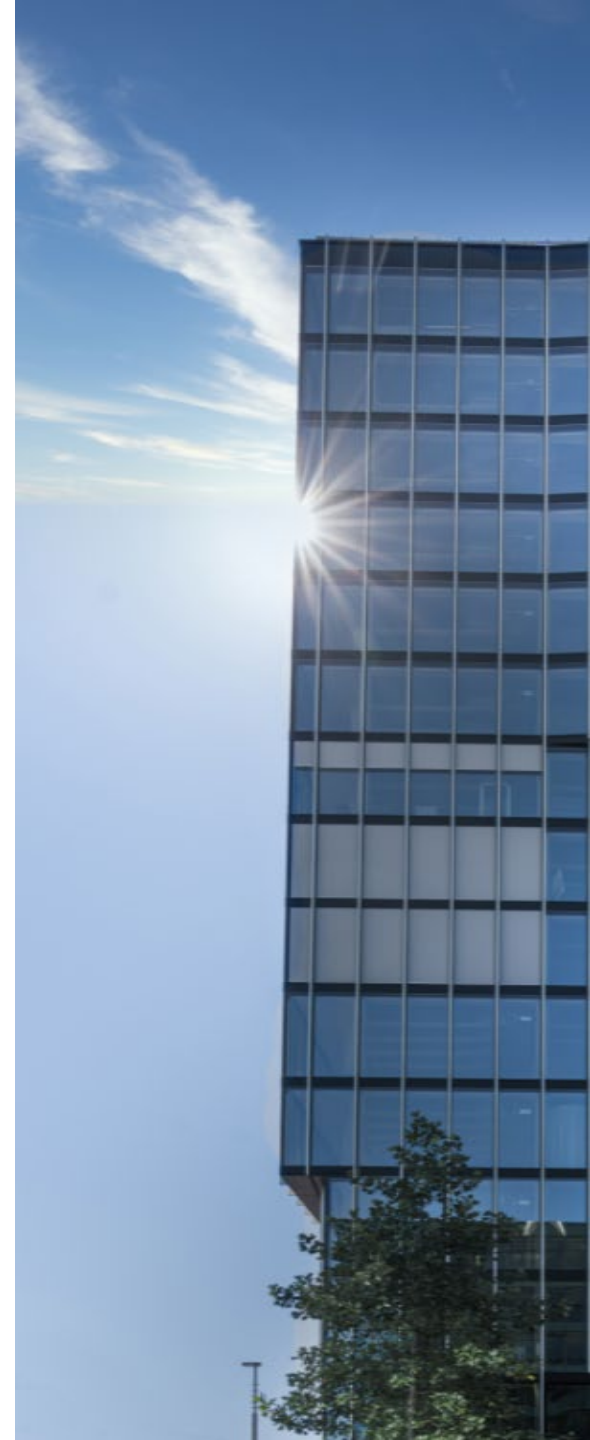


Objektorientierte Programmierung

GUI-Programmierung mit Java

Roland Gisler



Inhalt

- Herausforderung: Plattformunabhängigkeit von Java
- Java Technologien: AWT, Swing und JavaFX
- Unterschiedliche Entwicklungsvorgehen
- Empfehlungen
- Zusammenfassung

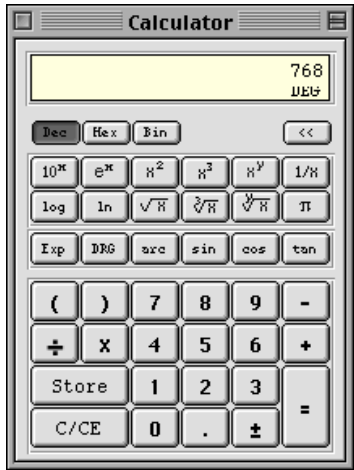
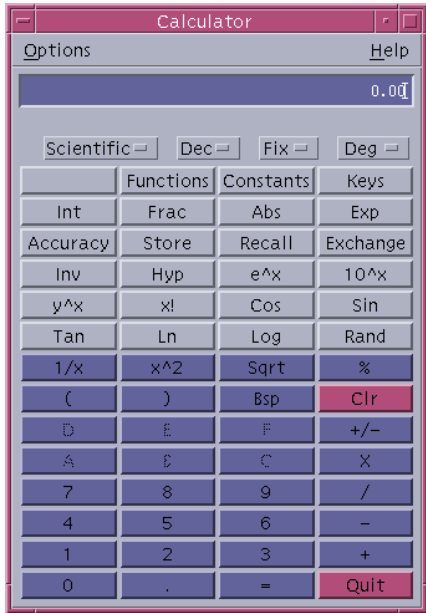
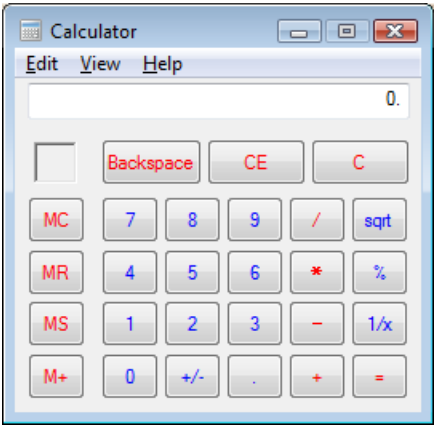
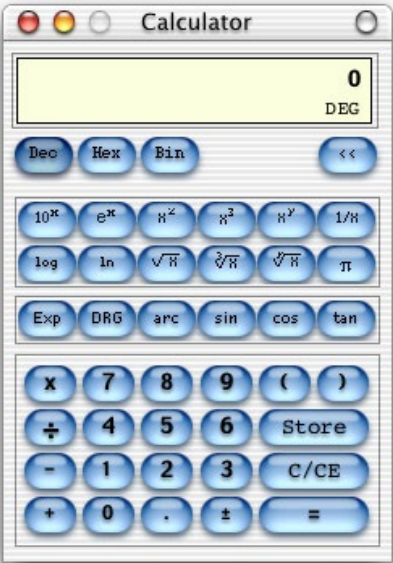
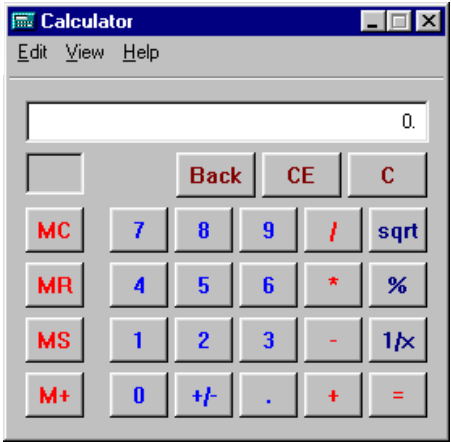
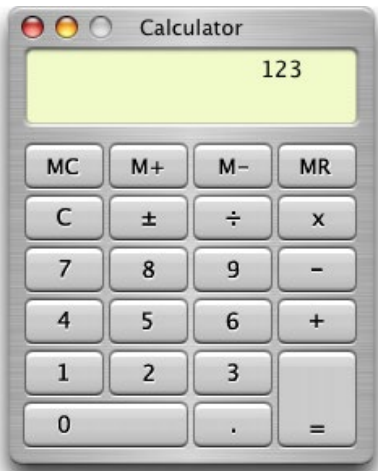
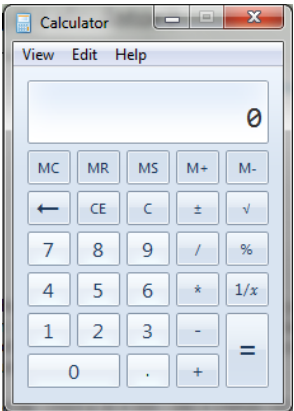
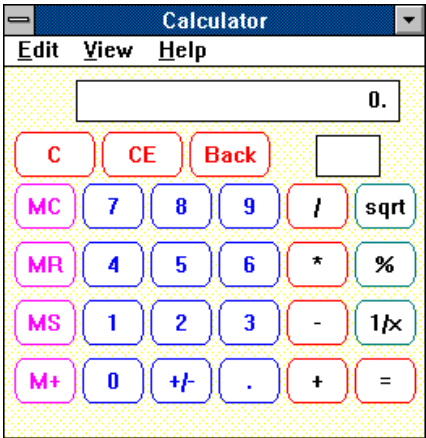
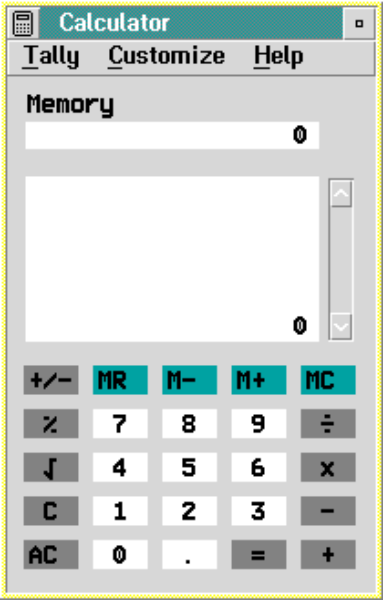
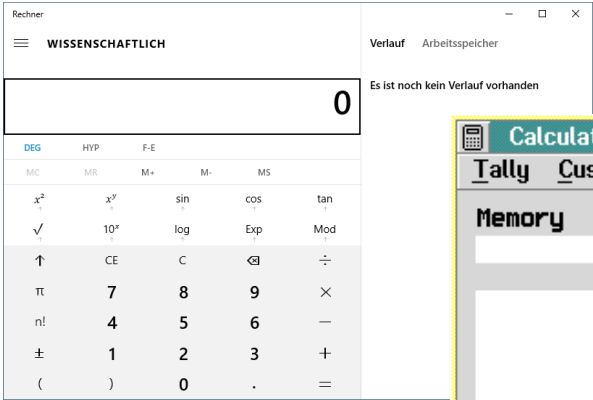
Lernziele

- Sie kennen die drei technologischen Ansätze für die GUI-Programmierung mit Java: AWT, Swing und JavaFX.
- Sie sind in der Lage, Code von einfachen GUI-Programmen zu verstehen und Änderungen daran vorzunehmen.
- Sie sind sich der Herausforderungen und Komplexität der GUI-Programmierung bewusst.

Plattformunabhängigkeit von Java

- Java ist plattformunabhängig und läuft dank der Java Virtual Machine (JVM) auf vielen verschiedenen OS-Plattformen.
 - Abstraktion des ganzen Betriebssystems mit Prozessen, Speicherverwaltung, Dateisystemen, I/O und auch des **GUI**!
- Im Bereich der grafischen Schnittstellen gibt/gab es zwischen den unterschiedlichen Betriebssystemen aber beinahe unüberwindbare Hürden!
- Es existieren unzählige, verschiedenste Window-Manager/Desktops:
 - Apple: Platinum, Aqua (verschiedene Generationen)
 - Unix/Linux: Motif, KDE, Gnome, Mate, Compiz,... (Dutzende!)
 - Windows: WPF (verschiedene Generationen)
- Wie bringt man das alles unter einen Hut?

Verschiedene Plattformen



GUI – Frameworks in Java

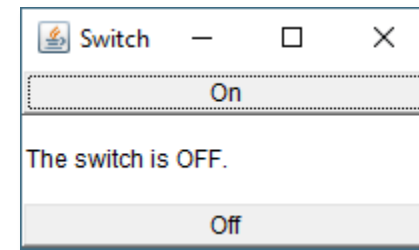
Java kennt mittlerweile **drei** verschiedene (und teilweise aufeinander aufbauende) GUI-Frameworks, die auch je in verschiedenen Generationen existieren:

- AWT
 - Seit Java 1.0 (1996)
- Swing
 - Seit Java 2 (1.2, 1998)
 - Weiterentwicklung inzwischen eingestellt (nur noch Wartung).
 - Derzeit (noch) mit Abstand die grösste Verbreitung.
- JavaFX
 - Seit Java 8 (2014) integriert, seit Java 11 (2018) externalisiert.
 - Wird seit 2008 entwickelt → modernstes Framework.
 - Jetzt OpenJFX (<https://openjfx.io/>), ist strikt modularisiert!

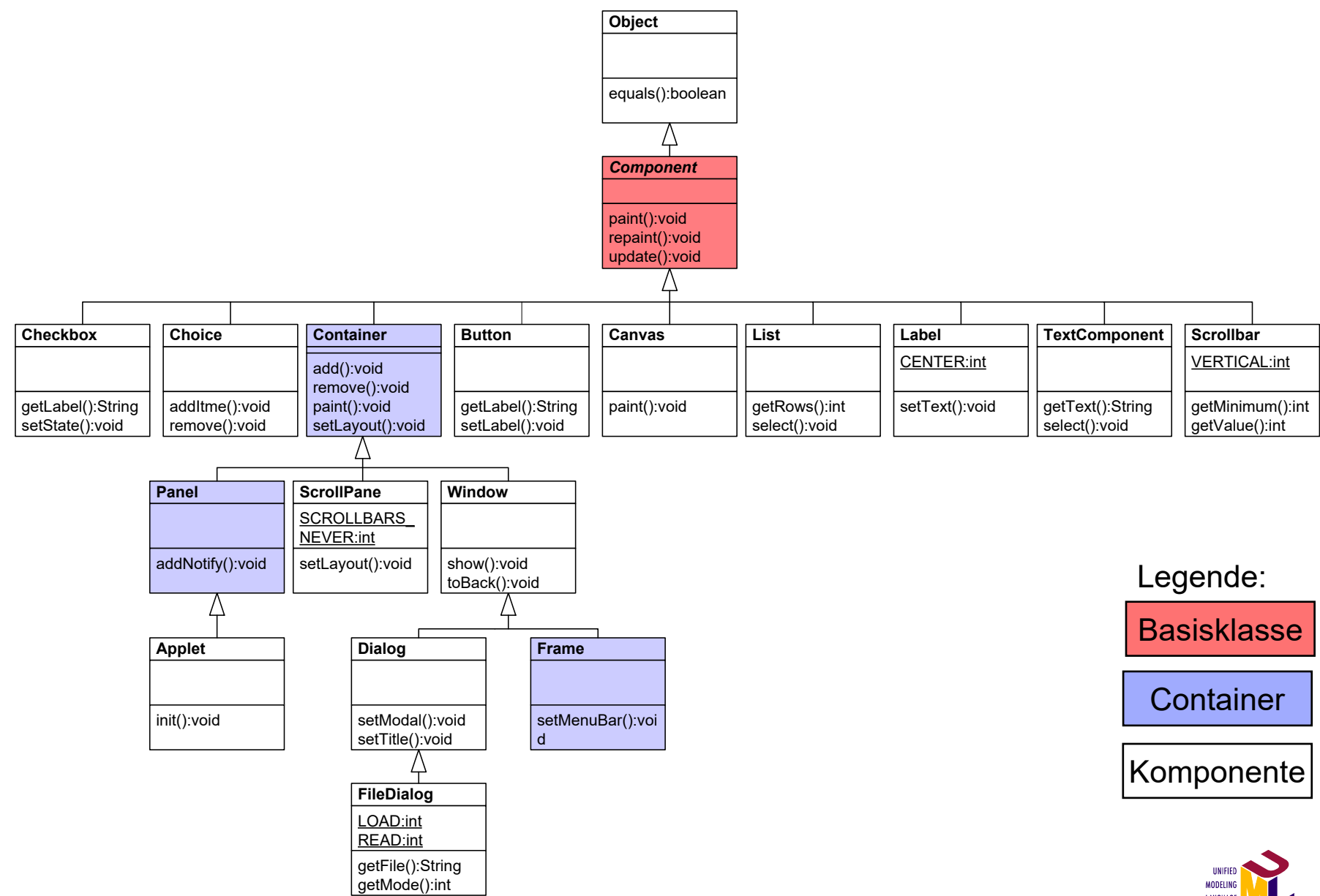
AWT (Abstract Window Toolkit)

Abstract Window Toolkit (AWT)

- AWT «wrapped» die nativen GUI-Komponenten des jeweiligen Betriebssystems!
 - Dadurch relativ schlank, einfach und effizient.
 - Look&Feel entspricht **voll** der jeweiligen Zielplattform.
- Hauptproblem: Weil Java-Programme auf **allen** Plattformen laufen müssen, wird nur die **kleinste gemeinsame Menge**, der auf allen Plattformen verfügbaren GUI-Komponenten unterstützt!
 - Somit nur sehr eingeschränkte Möglichkeiten.
- Auf einzelnen Plattformen fehlen evtl. für diese sehr typische und essentielle Komponenten!
 - Minimalistische Möglichkeiten.



AWT-Klassen (Auszug)

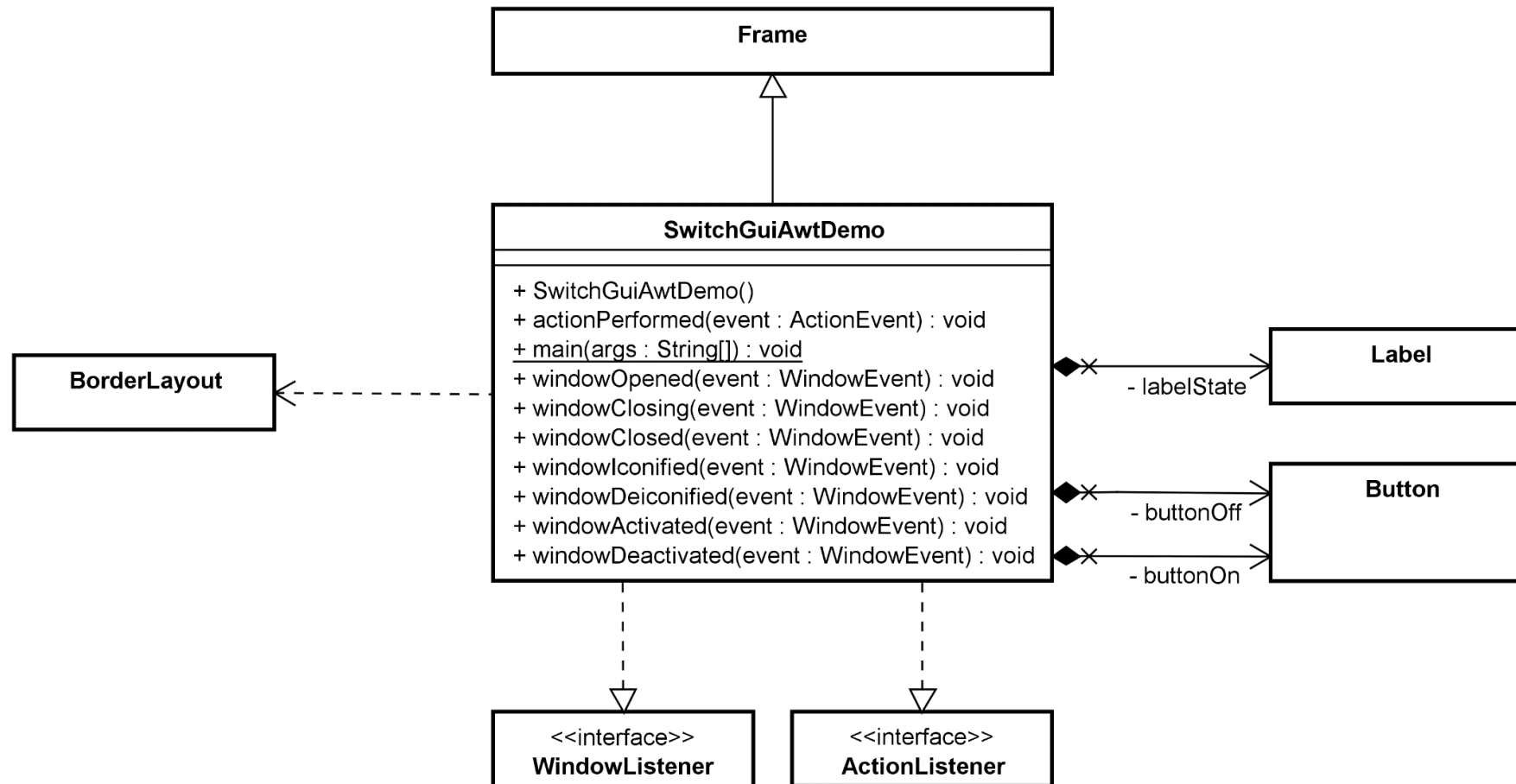


AWT – Die Basistechnologie im Hintergrund

- AWT stellt als solches zwar noch immer eine wichtige Basistechnologie für die GUI-Programmierung in Java dar, hat aber als API (Schnittstelle für Anwendungsentwicklung) für neue GUIs **keine** Bedeutung mehr.
- Klartext: Programmieren Sie **keine** neuen GUIs nur auf Basis von AWT, sondern verwenden Sie die darauf aufbauenden GUI-Frameworks wie **Swing** (alt) oder **JavaFX** (neu).

Demo

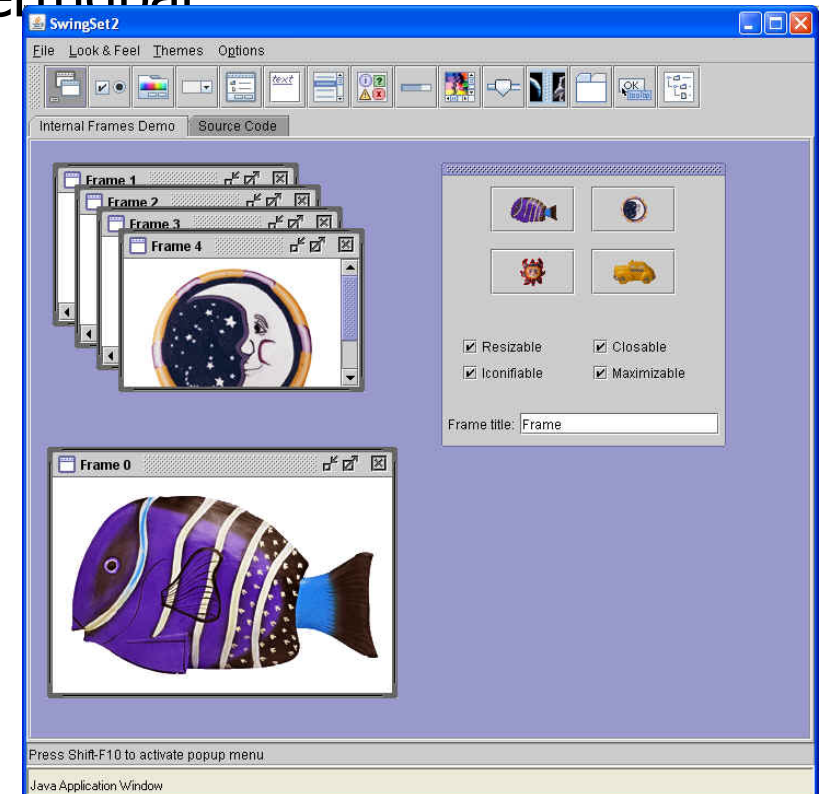
■ Java GUI mit AWT (Abstract Window Toolkit)



Swing

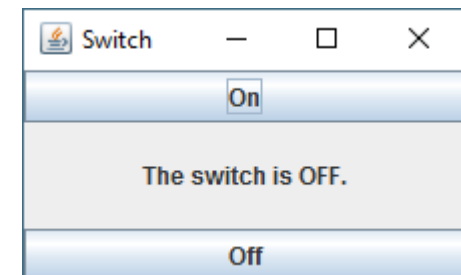
Swing – die Anfänge

- Mit Java 2 (1.2, 1998) wurde ein komplett neues GUI-Konzept realisiert: Java implementiert ein **eigenes Set** von umfangreichen, leichtgewichtigen GUI-Komponenten.
 - Quasi eine eigene Oberfläche, mit eigenem Look&Feel (Metal).
 - Auf **allen** Plattformen sind somit alle Komponenten verfügbar
 - Basiert auf wenigen, fundamentalen AWT-Klassen.
- Java-Applikationen sahen auf allen Plattformen identisch aus (ausser die Fensterdekoration selber) und waren darum klar als Java-Applikation erkennbar.
 - Der Metal-Style war sehr umstritten.
 - Benutzer*innen verlangten bald nach echtem OS-Look&Feel (existiert).

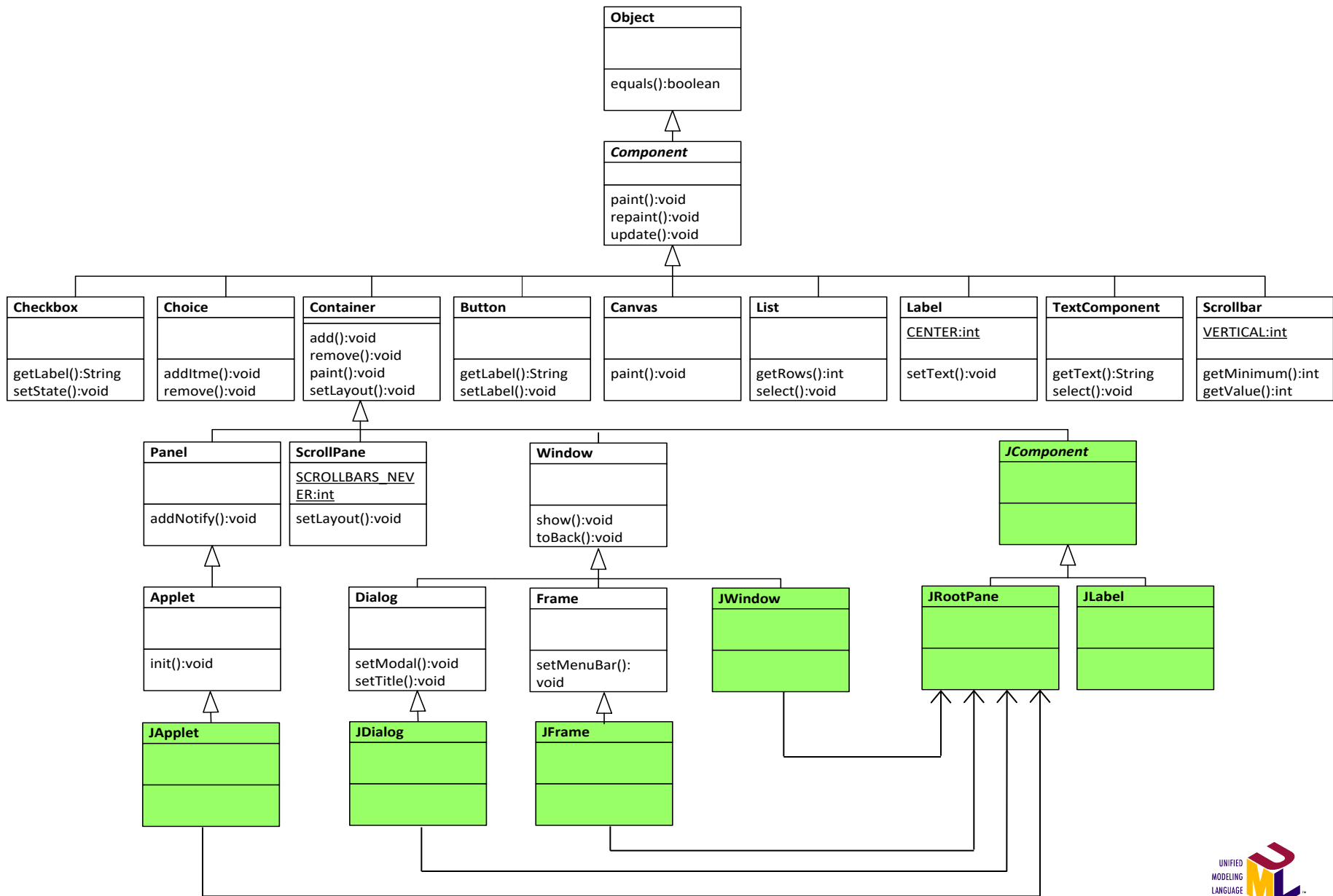


Swing – die Weiterentwicklung

- Bis und mit der Version Java 7 (1.7, 2011) wurde Swing sukzessive weiterentwickelt und verbessert.
 - Verbesserung der API, mehr Funktionalität.
 - Immer wieder neue Komponenten.
 - Nutzung des M(VC)-Prinzips (von Anfang an).
- Ein Meilenstein waren die (austauschbaren) Look&Feels, mit welchen die Darstellung der Komponenten derjenigen der nativen GUI-Komponenten nachempfunden wurde.
 - Damit ist es auch heute noch möglich, z.B. auf Windows den Motif-Style (Unix) zu aktivieren!
- Beispiel: Die Entwicklungsumgebung NetBeans basiert z.B. vollständig auf Swing-Technologie.



Swing-Klassen: Auszug, grüne Klassen

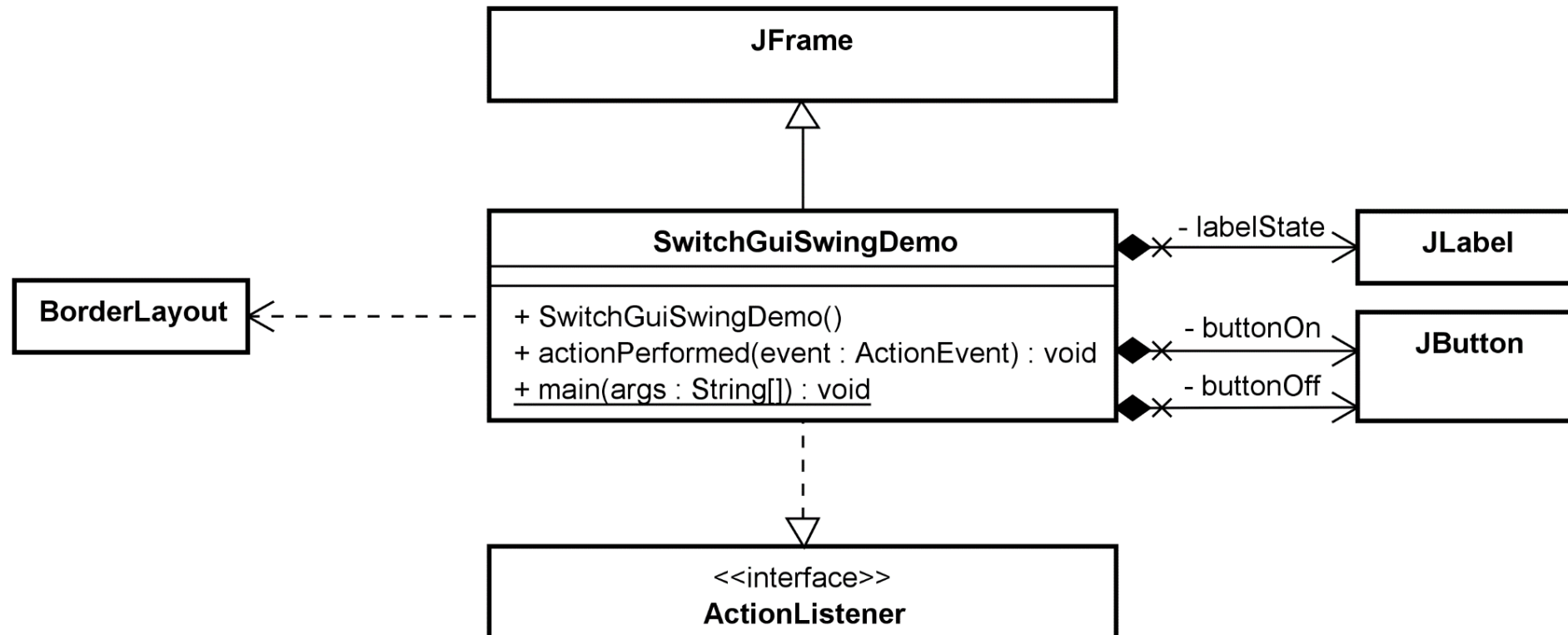


Swing – Aktueller Stand

- Swing ist derzeit noch immer die am meisten verwendete GUI-Technologie für Java-Applikationen.
 - Grund: Es existieren noch viele bestehende Applikationen.
- Vor wenigen Jahren wurde die Weiterentwicklung von Swing (zugunsten des GUI-Frameworks JavaFX bzw. OpenJFX) eingestellt.
 - Swing wird aber noch immer aktiv gewartet.
- Das hat viele Anwender*innen von Java stark verunsichert, da JavaFX relativ lange benötigte, bis es einen brauchbaren, produktionsreifen Stand erreichte.
 - Seit Java 8 ist das aber gegeben.
- Konkret: Während bestehende Applikationen zumeist auf Swing basieren, ist für Neuentwicklungen in der Regel JavaFX klar vorzuziehen.

Demo

■ Java GUI mit Swing



JavaFX

JavaFX

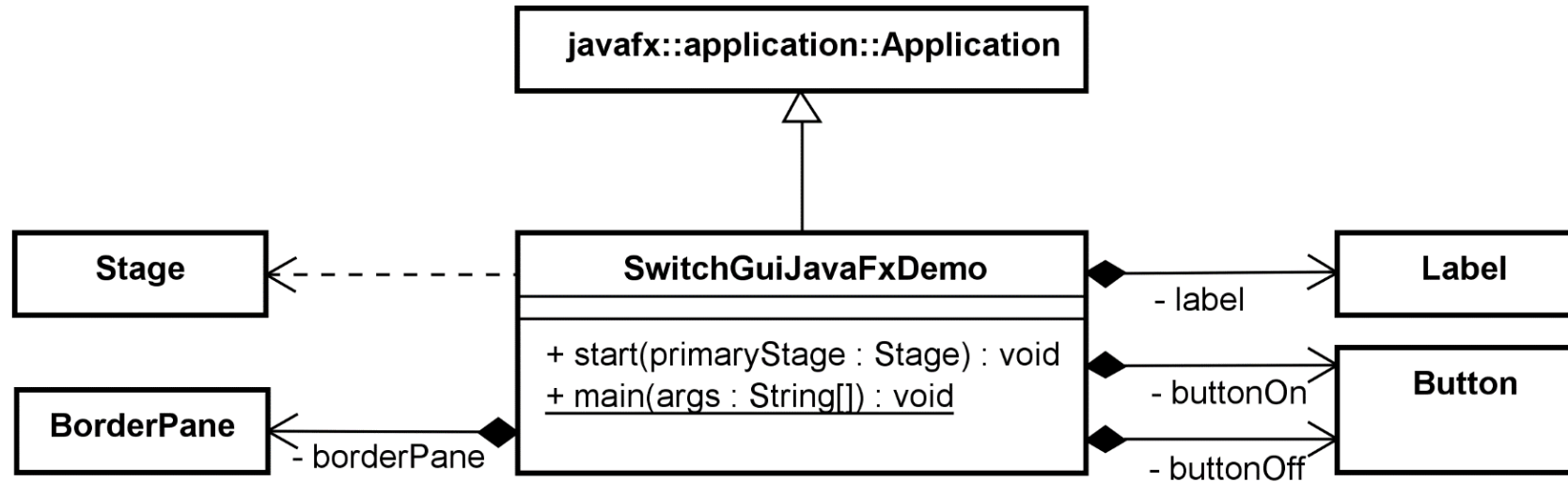
- Eigenwerbung: Oracle's neues GUI-Framework für plattformübergreifende „Rich Applications“.
- Wie in Swing wurden eigenständige Komponenten entwickelt.
 - Die Komponenten wurden gegenüber Swing aber stark erweitert und die API ist deutlich stärker mehr „Nutzergetrieben“, d.h. es ist einfacher zu Verwenden.
- Zusätzlich wurde das Programmiermodell selber stark modernisiert und vereinfacht.
 - Wesentlich konzeptioneller, verwendet z.B. **Stage** und **Scenes**.
- Es ist möglich, GUI's per FXML (→ SceneBuilder) zu deklarativ zu entwerfen.
 - Vergleichbar mit XUL und XAML (GUI-Beschreibungssprachen).
 - Somit grosses Potential für externes Design, Ergonomie etc.
- Die Darstellung kann einfach mittels Stylesheets (CSS) angepasst werden.
 - Dafür ist keine Codeänderung und auch keine Neukompilierung notwendig.

Grundlagen von JavaFX

- JavaFX ist im Vergleich zu Swing ein echtes Framework, weil es den Aufbau eines GUIs deutlich mehr abstrahiert, vereinfacht und automatisiert.
- Konzept von **Stage**, **Scene**, **Node** und Scene Graph, stark vereinfacht:
 - **Stage** ist eine Analogie zum Frame (Window).
 - **Scene** ist eine Analogie zum (Root-)Panel (Content).
 - **Node** ist die Basisklasse für alle GUI-Elemente / Komponenten.
 - Scene Graph ist eine (logische) hierarchische Datenstruktur, welche alle Nodes enthält.
- JavaFX übernimmt das Rendering (Darstellung) des ganzen Scene Graph vollständig.
 - Erlaubt z.B. die sehr einfache Realisation von Animationen.

Demo

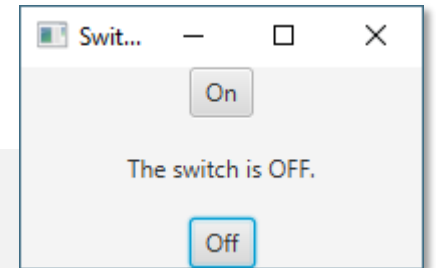
- Java GUI mit JavaFX (inkl. FXML)



JavaFX – FXML (und CSS)

- Ebenfalls herausragend an JavaFX ist, dass es damit (für Java) erstmals möglich ist, das Design der Visualisierung von der eigentlichen Programmierung vollständig zu trennen: FXML
- FXML (**FX-XML**) ist eine auf XML basierende Definition für User-Interfaces (vergleichbar mit XUL und XAML).
- Beispiel:

```
<?xml version="1.0" encoding="UTF-8"?>
...
<BorderPane prefHeight="100.0" prefWidth="200.0" ... >
  <top>
    <Button fx:id="btnOn" mnemonicParsing="false" text="On" BorderPane.alignment="CENTER" />
  </top>
  <bottom>
    <Button fx:id="btnOff" mnemonicParsing="false" text="Off" BorderPane.alignment="CENTER" />
  </bottom>
  <center>
    <Label fx:id="label" text="The switch is OFF." BorderPane.alignment="CENTER" />
  </center>
</BorderPane>
```



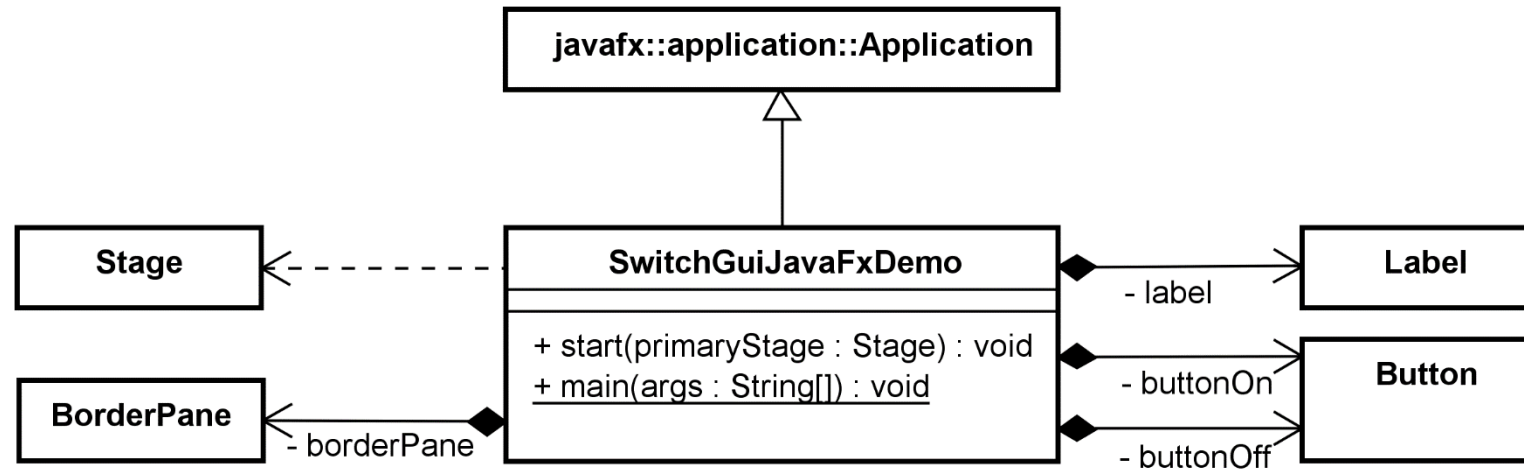
JavaFX – Verbindung von FXML mit Code

- Die Verbindung zwischen FXML und dem Code (welcher z.B. die Events verarbeitet) geschieht über «**fx:id**»: Einfache **String**-Bezeichner, die als Identifikation dienen.
 - Fluch und Segen: Einfache Tippfehler wirken sich fatal aus!
- Die Programmierung wird vereinfacht, in dem so genannte → Dependency Injection verwendet wird, um die vom JavaFX-Framework aus der FXML-Beschreibung erzeugten GUI-Elemente in die Controller-Objekte zu setzen (Setter-Methoden).
 - @FXML-Annotationen für Attribute.

```
public final class SwitchGuiJavaFXMLController implements Initializable {  
  
    @FXML  
    private Button btnOn;           // fx:id="btnOn"  
  
    @FXML  
    private Button btnOff;         // fx:id="btnOff"  
    ...  
}
```

Demo

- siehe Slide [21](#)



Starten von JavaFX-Applikationen

Starten von JavaFX-Applikationen

- JavaFX wurde aus dem JDK ausgegliedert (eigener Release).
 - Grundsätzlich eine gute Idee, das JDK wird dadurch schlanker bzw. nicht noch fatter!
- JavaFX nutzt konsequent die neusten Java-Technologien.
 - Auch die mit Java 9 eingeführte (aber leider noch immer nicht breit etablierte) Technologie zur → Modularisierung (**J**ava **P**latform **M**odul **S**ystem, JPMS).
- JavaFX basiert auf plattformspezifischen Binaries, welche entsprechend vorliegen müssen. Vergleichbare Technologie: **S**tandard **W**idget **T**oolkit (SWT) von Eclipse.
- Konsequenzen: Das Starten (und Entwickeln) von JavaFX-Applikationen ist im Vergleich zum Start von AWT oder Swing (die komplett in Java integriert sind) etwas komplizierter!
 - Eine gute Anleitung finden sie unter <https://openjfx.io/openjfx-docs/>
 - Einfacher geht es während der Entwicklung z.B. mit Maven: `mvn javafx:run`
 - Bedingung: Die Startklasse muss im `pom.xml` eingetragen sein.

Vorgehen: Programmieren oder GUI-Designer?

Variante 1: Reine Programmierung

- Klingt ein bisschen nach «hardcore»:
Ein GUI von A bis Z zu programmieren, ist sehr aufwändig.
- Tatsächlich ist der Initialaufwand sicherlich deutlich höher.
 - Er sinkt aber mit einer guten Vorbereitung: Die Planung, wie das GUI aufgebaut und strukturiert werden soll, ist ohnehin essentiell!
- Der Code ist in der Regel gut verständlich und effizient, dadurch sind spätere Änderungen und Korrekturen leichter möglich.
- Hauptnachteil: «Nur» Programmierer*innen können das GUI verändern, reine UX-Designer*innen sind in der Regel total überfordert.
- Ein wichtiges Kriterium: Wie «dynamisch» (Häufigkeit) muss sich ein GUI an neue Anforderungen anpassen können?

Variante 2: GUI-Designer

- Swing:

- GUI-Designer, welcher direkt Java-Code erzeugt, z.B. in NetBeans. Verleitet leider stark zu «Quick&Dirty»-GUIs (ohne Planung), die schlecht wartbar sind.
- Code meist schwer verständlich, Anpassungen häufig nur noch über GUI-Designer möglich, somit Vorsicht vor starker Abhängigkeit zum Produkt / der IDE.

- JavaFX / OpenJFX:

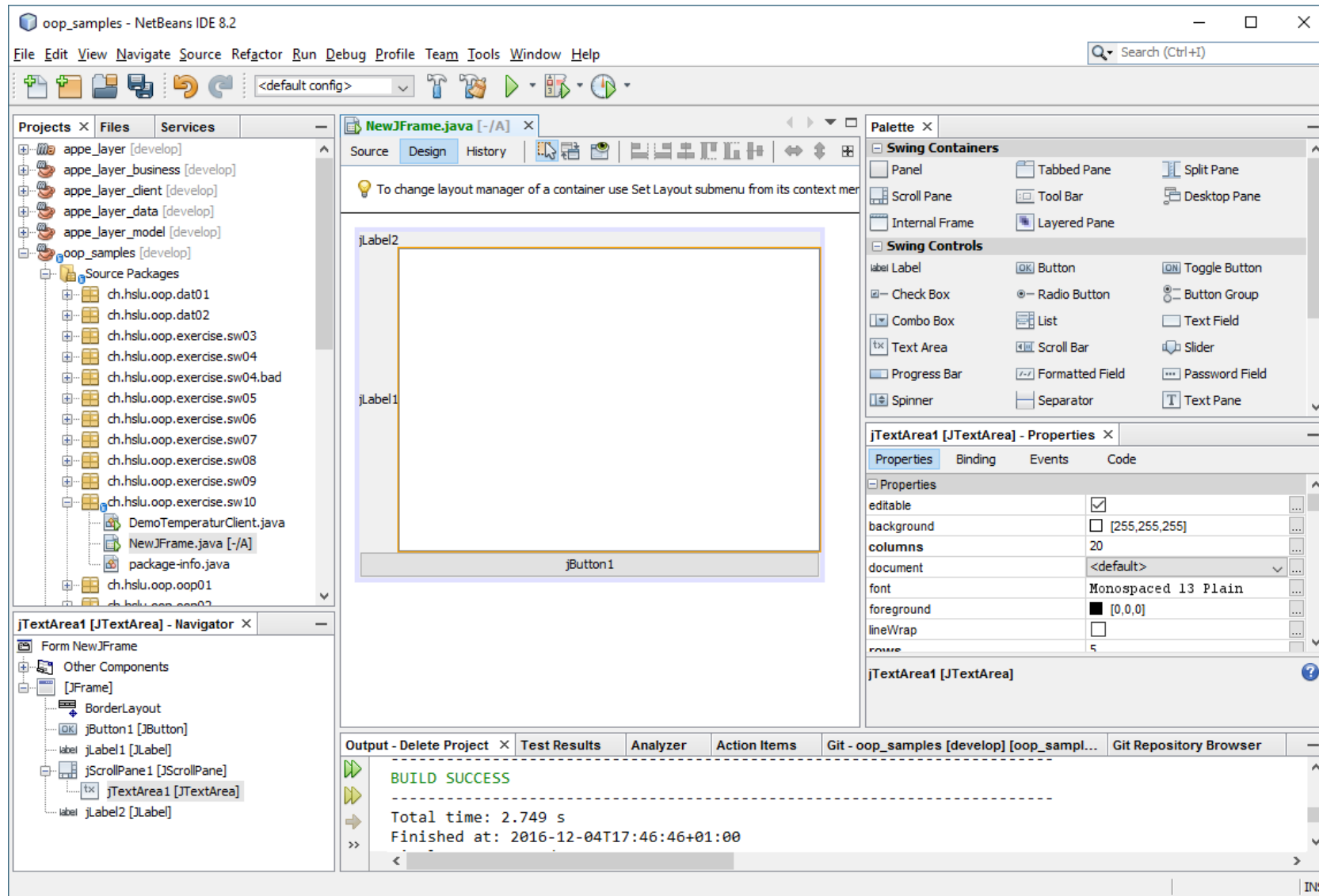
- GUI-Definition technologisch über FXML weitgehend getrennt vom Code: Zwei (sinnvoll) getrennte Welten, erfüllt SRP.
- Schwäche: Binding zw. FXML und Code per String-IDs ist weniger robust.

- Bilanz: Situation für Swing und JavaFX **nicht** identisch.

- Für Code haben wir die Entwicklungsumgebung.
- Für GUI-Design haben wir GUI-Editoren (z.B. SceneBuilder).

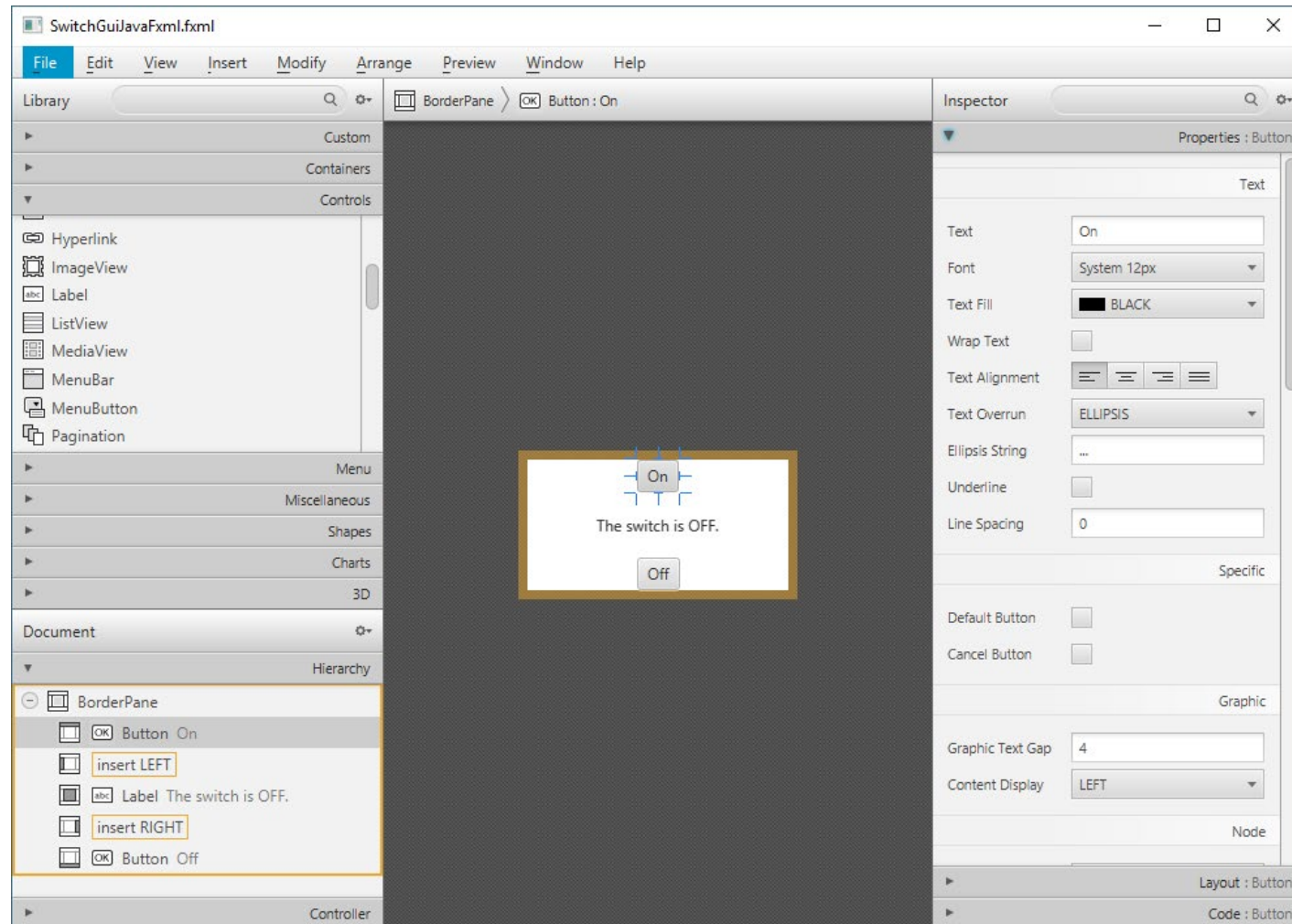
➔ Somit JavaFX mit GUI-Designer klar im Vorteil?

GUI-Designer für Swing – Beispiel Netbeans



GUI-Designer für JavaFX – FXML-Authoring

- Gluon Scene Builder - <http://gluonhq.com/labs/scene-builder/>



Empfehlungen



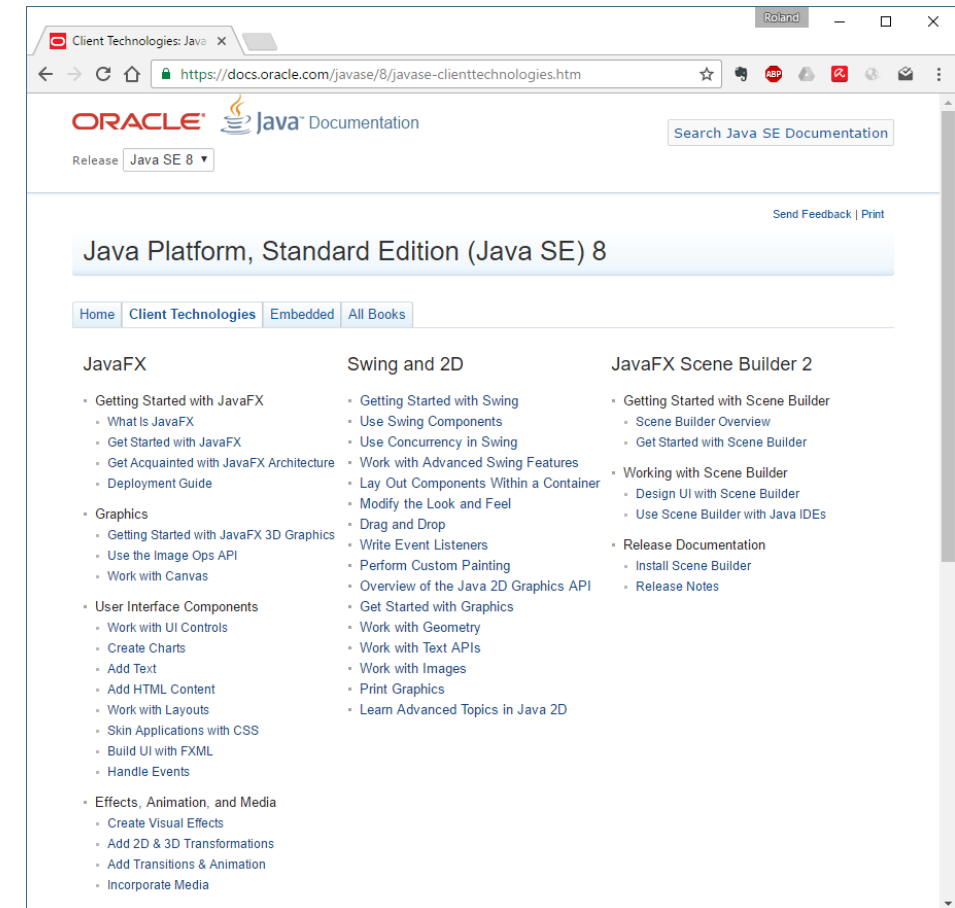
Empfehlungen zur GUI-Programmierung

- GUI-Programmierung ist aufwändig, darum ist eine gute Vorarbeit notwendig:
GUI-Mockup für Design, Strukturierung, Anordnung, Ergonomie, saubere Trennung der Aufgaben (MVC) → Viel Fleissarbeit!
- Neue Java-GUIs: Unbedingt mit JavaFX (zukunftsgerichtet).
 - Viele bestehende GUIs basieren aber noch auf Swing.
- Schwieriger Grundsatzentscheid:
Explizites Programmieren oder mit Hilfe von GUI-Designer?
 - Es gibt viele Argumente dafür und dagegen.
 - Nicht selten spaltet dieser Entscheid die Entwicklergemeinde / das Team.
 - Mittelfristig wird der GUI-Designer (im Zusammenhang mit «neuen» Technologien wie FXML und CSS) aber gewinnen.
- Ergonomie und Usability nicht vergessen!

Einstieg in Java GUI Programmierung



- GUI-Programmierung ist ein wirklich **sehr** umfangreiches Thema, das – wenn man es erschöpfend behandeln möchte – problemlos ein ganzes Modul / Semester befüllen könnte.
- Wenn Sie sich in das Thema vertiefen wollen, empfehle ich Ihnen als erstes die Dokumentation von Oracle, welche auch verschiedenste Tutorials enthält.
- Mehr Informationen finden Sie auf der Seite <https://openjfx.io/>
- siehe: <https://docs.oracle.com/javase/8/javase-clienttechnologies.htm>



Zusammenfassung

- GUI-Programmierung ist ein sehr umfangreiches Thema.
- AWT als grundlegende Technik.
- Auswahl der Technologie/Framework: Swing oder JavaFX.
- Entwicklungsvorgehen: Immer zuerst einen Entwurf und eine Planung, erst dann Umsetzung – egal ob mit GUI-Designer oder native Programmierung.



Fragen?